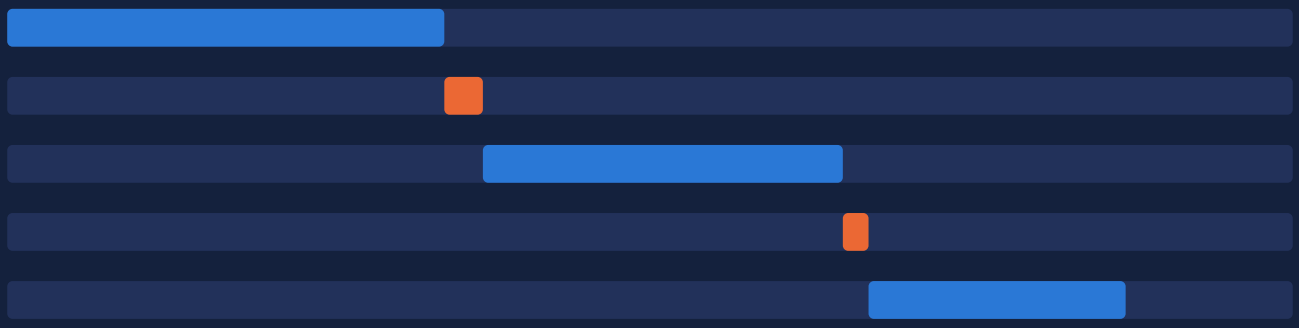
AgentLens

A beginner's guide to AI agents,
observability, evaluation and regression testing



What the project does, how it works, and why each part matters —
explained from zero, no AI background needed.

September 2026

Contents

How to read this guide . . . 4

Part 1 — AI from zero . . . 5

1. What is a language model? . . . 6
2. Tokens: the unit AI is measured in . . . 7
3. Prompts, messages and roles . . . 7
4. Why AI is unpredictable and sometimes makes things up . . . 8
5. From chatbot to agent: tools . . . 9
6. APIs, providers and rate limits . . . 11

Part 2 — The problem AgentLens solves . . . 12

7. Why agents are hard to trust . . . 13
8. Idea one: observability (the flight recorder) . . . 13
9. Idea two: evaluation (grading the answers) . . . 14
10. Idea three: regression testing . . . 14

Part 3 — How AgentLens works . . . 16

11. The big picture . . . 17
12. The sample world: ShopWise . . . 18
13. The agent . . . 18
14. Tracing: recording everything . . . 20
15. Storing traces: the database . . . 22
16. The evaluators, one by one . . . 23
17. The test dataset and experiments . . . 25
18. Regression detection . . . 25
19. The backend API (FastAPI) . . . 27
20. The dashboard (Next.js) . . . 27
21. Docker: packaging it up . . . 29
22. Testing without an AI . . . 30

Part 4 — Using AgentLens . . . 31

23. Setting it up . . . 32
24. The everyday workflow . . . 33
25. How to read a run . . . 33

26. Troubleshooting 34

Part 5 — Reference 35

27. Limitations 36

28. Future improvements 36

29. How this relates to professional tools 36

30. Glossary 38

31. Code map and command cheat sheet 40

What you now understand 41

How to read this guide

This guide explains AgentLens: a small software project that watches an AI agent work, grades how well it did, and warns you when a new version of the agent got worse. You don't need any background in artificial intelligence to follow it. We start from what an AI model actually is and build up, one idea at a time, until every part of the project makes sense.

The guide has five parts:

- **Part 1 — AI from zero.** What a language model is, what tokens and prompts are, why AI makes things up, and how a chatbot becomes an *agent* that can use tools.
- **Part 2 — The problem.** Why agents are hard to trust, and the three ideas that help: *observability*, *evaluation* and *regression testing*.
- **Part 3 — How AgentLens works.** A guided tour through every piece of the project, following one real question from start to finish.
- **Part 4 — Using it.** Setting it up, running experiments, and reading the dashboard.
- **Part 5 — Reference.** Limitations, future ideas, a glossary of every term, and a map of the code.

Throughout the guide you'll see coloured boxes:

> **Everyday analogy**
> Compares a technical idea to something from everyday life.

> **Why this matters**
> Explains what would go wrong without this idea.

> **In AgentLens**
> Points to where the idea lives in the AgentLens code.

> **Watch out**
> A common mistake or a limitation to keep in mind.

Code snippets are short excerpts from the real project files. Example outputs that depend on a live AI model are marked as *illustrative*: your numbers will differ, because AI models don't give exactly the same answer every time.

PART 1

AI from zero

Before we can observe or grade an AI agent, we need to know what one is. This part covers the handful of ideas everything else is built on: language models, tokens, prompts, hallucinations, tools and the agent loop.

1. What is a language model?

When people say "AI" today they usually mean a **Large Language Model**, or **LLM**. ChatGPT, Claude, Gemini and Llama are all LLMs. Despite how clever they seem, the core job of an LLM is surprisingly simple: **given some text, predict what text comes next.**

An LLM is trained by showing it an enormous amount of written text (books, websites, code) and repeatedly asking it to guess the next word. Every wrong guess nudges millions of internal numbers, called *parameters* or *weights*, a tiny bit in the right direction. After enough practice the model becomes extremely good at predicting text, and to predict text well it has to pick up grammar, facts, reasoning patterns and even how to write code.

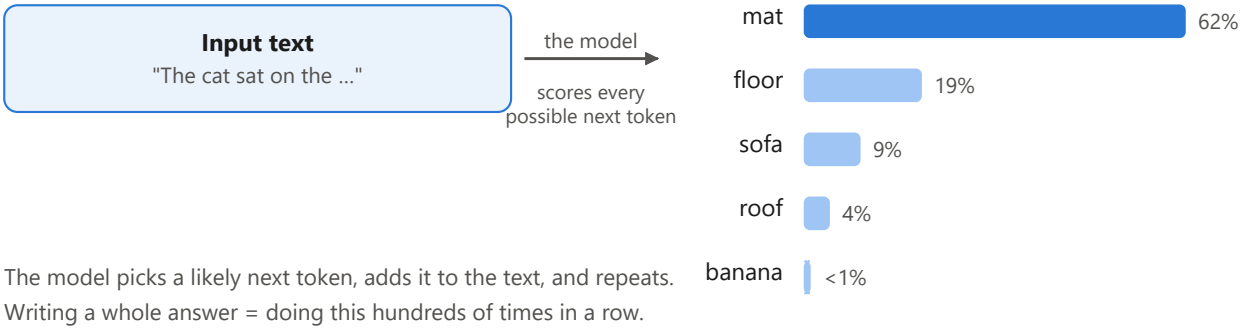


Figure 1 — An LLM turns text into a ranked list of likely next pieces of text.

> **Everyday analogy**
> Your phone keyboard suggests the next word as you type. An LLM is the same idea, trained on vastly more text and with billions of parameters, so its "suggestions" can be whole, sensible paragraphs.

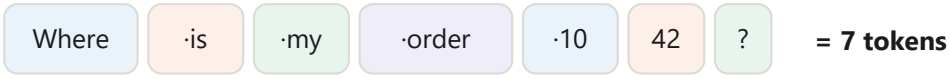
Two consequences of this design matter a lot for AgentLens:

- **An LLM produces plausible text, not verified text.** It writes what sounds right. Usually that's also correct, but not always.
- **An LLM only knows what was in its training data and what you put in front of it.** It has never seen your company's order database, so it can't know where order 1042 is unless something shows it.

The number in a model's name often hints at its size. In this project we use two open models served by a company called Groq: **gpt-oss-120b** (about 120 billion parameters) and **gpt-oss-20b** (about 20 billion). Bigger models are usually smarter but slower and more expensive to run. Choosing between them is a real trade-off, and measuring that trade-off is one of the things AgentLens is for.

2. Tokens: the unit AI is measured in

LLMs don't read letters or whole words. They read **tokens**: small chunks of text, typically a word, part of a word, or a punctuation mark. A rough rule of thumb for English is that one token is about ¾ of a word, so 100 tokens is roughly 75 words.



Tokens are chunks of text, often a word or part of a word (· marks a space). Exact splits depend on the model.

Figure 2 — A sentence split into tokens (illustrative split).

Tokens matter because **almost everything about using an LLM is counted in tokens**:

What	How tokens are involved
Cost	Providers charge per token, separately for **input tokens** (what you send) and **output tokens** (what the model writes). Output is usually several times more expensive.
Speed	The model generates output one token at a time, so longer answers take longer.
Limits	Each model has a maximum *context window* (how many tokens it can read at once), and providers cap how many tokens you may use per minute or per day.

Prices are usually quoted **per million tokens**. For example, Groq's published price for gpt-oss-120b is $0.15 per million input tokens and $0.60 per million output tokens (as of September 2026). One agent run might use about 1,100 input tokens and 55 output tokens:

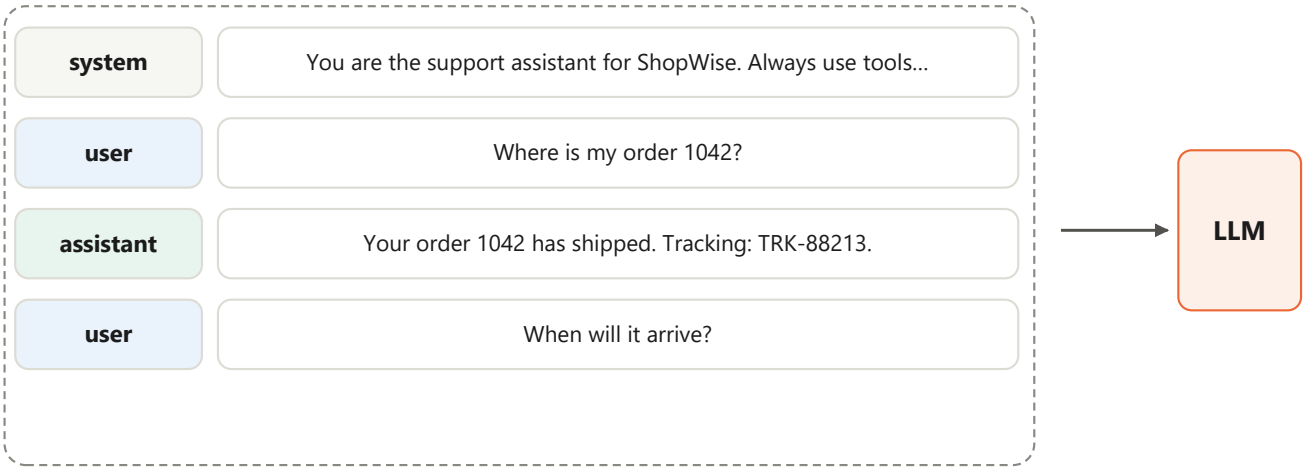
```
cost = 1,100 × $0.15 / 1,000,000  +  55 × $0.60 / 1,000,000
     = $0.000165              +  $0.000033
     = $0.000198   (about 2 hundredths of a cent)
```

> **Why this matters**
> A fraction of a cent sounds like nothing, but a product answering 100,000 questions a day multiplies it by 100,000. A new prompt that doubles token usage doubles the bill. That is why AgentLens records tokens and estimated cost for every single run.

3. Prompts, messages and roles

The text you send to an LLM is called the **prompt**. Modern chat models accept the prompt as a list of **messages**, each with a **role**:

Role	Who writes it	Purpose
system	The developer	Standing instructions: who the assistant is, rules it must follow, today's date.
user	The person asking	The actual question or request.
assistant	The model	The model's earlier replies (including requests to use tools).
tool	Your code	The result of a tool the model asked for (explained in section 5).



The WHOLE conversation is sent on every call. The model itself remembers nothing between calls.

Figure 3 — A conversation is a list of messages sent to the model all at once.

A surprising fact: **the model has no memory between calls.** It feels like a chatbot remembers your earlier messages, but really the application re-sends the entire conversation every time. Each new turn makes the prompt longer, which means more input tokens, more cost and more time.

The system prompt has a huge effect on behaviour. In AgentLens, the two versions of our agent differ mainly in their system prompt. Version 1 has detailed rules ("always look up the order, never guess"). Version 2 has one short sentence. Small changes in wording can change results in ways that are hard to predict, which is exactly why we need to measure them.

4. Why AI is unpredictable and sometimes makes things up

Non-determinism

Remember that the model produces a ranked list of possible next tokens. The application can always pick the top one, or sample with some randomness to make text more varied. That randomness is controlled by a setting called **temperature**: 0 means "almost always pick the most likely token", and higher values mean more variety. AgentLens uses temperature 0, but even then providers don't guarantee identical output every time. Ask the same question twice and you may get two slightly different answers.

> **Why this matters**
> Normal software is predictable: 2 + 2 always returns 4, so one test proves it works. An AI agent may pass a test today and fail it tomorrow. That means we have to test many cases, look at percentages instead of single results, and keep a history.

Hallucination

A **hallucination** is when a model states something false with total confidence, for example inventing an order date or a refund policy. It isn't lying. It is doing its one job, producing plausible text, in a situation where the plausible text happens to be wrong. Hallucinations are most likely when the model is asked about facts it was never shown.

The main cure is **grounding**: give the model the real facts (from a database or document) and instruct it to answer only from those. A grounded answer is also called **faithful** to its sources. AgentLens checks faithfulness automatically, as you'll see in Part 3.

5. From chatbot to agent: tools

A plain chatbot can only answer from what it already knows. An **AI agent** is an LLM that can also **take actions**: look up data, call a calculator, search documents, send an email. These actions are called **tools**.

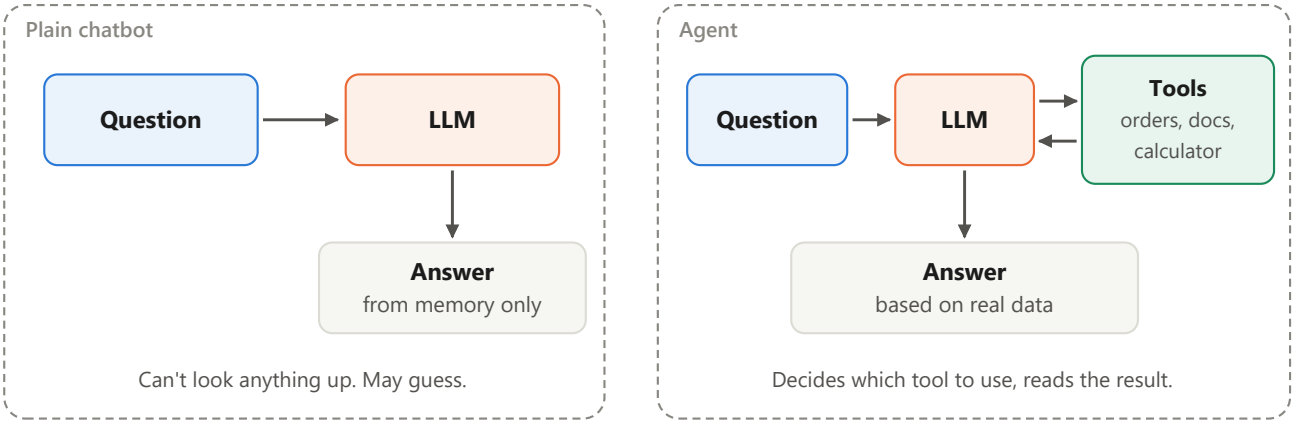


Figure 4 — The difference between a chatbot and an agent is access to tools.

How tool calling works

The LLM can't run code by itself. Instead, a process called **tool calling** (or **function calling**) happens:

1. Along with the conversation, your program sends the model a **list of available tools**. Each one has a name, a description and the inputs it expects, written in a structured format called a *JSON schema*.
2. Instead of answering in plain text, the model can reply with a structured request: `call lookup_order with {"order_id": "1042"}`.
3. **Your program** runs the real Python function, gets the result, and adds it to the conversation as a *tool* message.
4. The model is called again. Now it can see the tool result and write a grounded answer, or ask for another tool.

> **Everyday analogy**
> Picture a new customer-support employee on their first day. They're articulate, but they don't know any customer's order. You give them a handbook (the help articles), access to the order system (a lookup tool) and a calculator. For each question they decide which resource to check, read it, and reply. The LLM is that employee. The tools are the resources. Your program is the office that actually runs the lookups.

The agent loop

Put tool calling in a loop and you have an agent. The loop keeps going until the model answers in plain text, or until a safety limit is reached.

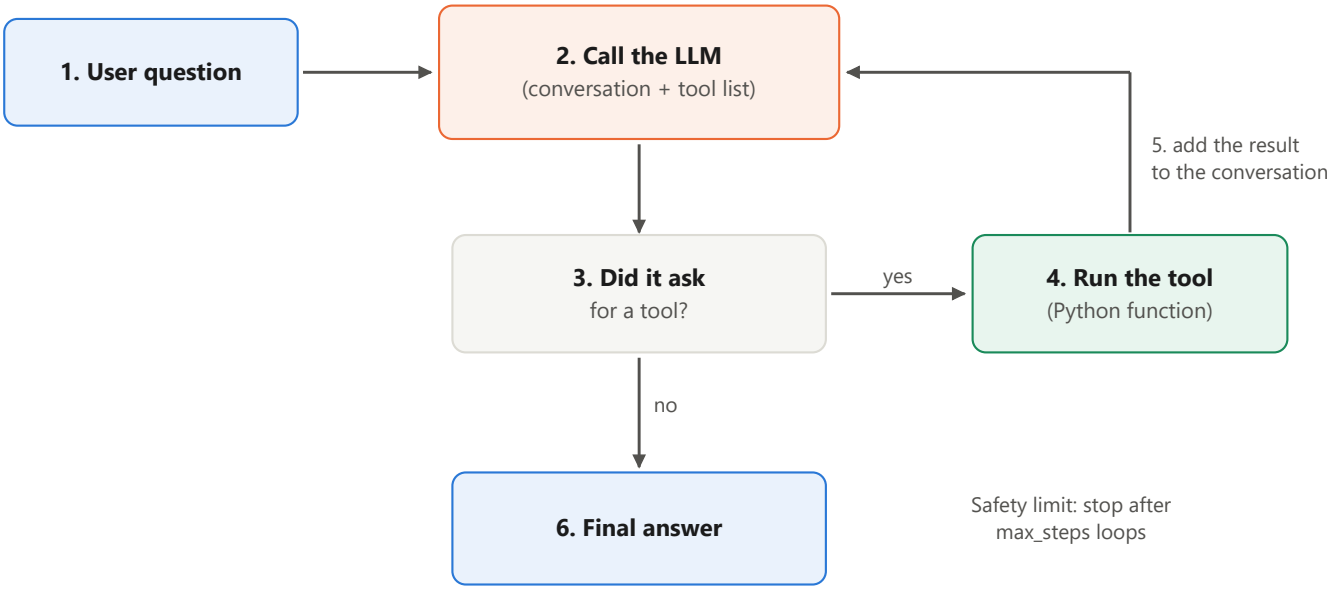


Figure 5 — The agent loop. Every agent framework is a version of this.

One question can take several trips around the loop. "Can I return the headphones from order 1041?" needs the order (to learn the delivery date) *and* the return policy (to learn the time limit) before the model can answer. Each trip is a **step**. The **max_steps** limit stops a confused model from looping forever and burning tokens.

> **In AgentLens**
> The whole AgentLens agent loop is about 30 lines in `backend/agent/support_agent.py`. Frameworks like LangGraph or CrewAI add features on top of this same loop. We wrote it by hand so that every step is visible, and visible steps are what AgentLens observes.

6. APIs, providers and rate limits

Running a large model needs powerful, expensive hardware (GPUs). Most developers don't run models themselves. They send requests over the internet to a **provider** such as OpenAI, Anthropic, Google or Groq. The provider runs the model and sends back the answer.

The way programs talk to each other over the internet is called an **API** (Application Programming Interface). You send a request containing the model name, the messages and the tool list, and you get back a response containing the model's reply and a **usage** section that says how many tokens were used. AgentLens reads that usage section to track tokens and cost.

Providers also enforce **rate limits** so no single user overwhelms their servers. Groq's free tier for our models allows roughly:

Limit	Free tier (per model)	What it means
Requests per minute	30	at most 30 calls to the model each minute
Tokens per minute	8,000	about 2–3 agent runs per minute before you must wait
Tokens per day	200,000	a few full experiments per day

When you go over a limit, the provider answers with an error called **HTTP 429 — Too Many Requests**, usually with a *retry-after* header saying how many seconds to wait. AgentLens handles this automatically, and does it in a way that doesn't distort its measurements (see Part 3, section 10).

PART 2

The problem AgentLens solves

An agent that works in a demo can fail in many quiet ways in real use. This part explains why agents are hard to trust, and introduces the three engineering ideas that make them trustworthy: observability, evaluation and regression testing.

7. Why agents are hard to trust

Traditional software follows exact instructions, so when it fails you can usually reproduce the failure and read the code. AI agents are different in four ways:

Property	What it means	Consequence
Black box	You can't read the model's "code"; its behaviour comes from billions of learned numbers.	You can only understand it by watching what it does.
Multi-step	One answer may involve several LLM calls and tool calls.	A mistake in step 2 can hide behind a fluent final answer.
Non-deterministic	The same input can give different outputs.	One successful test proves little; you need many.
Fluent when wrong	Wrong answers sound just as confident as right ones.	Users and developers can't tell by reading.

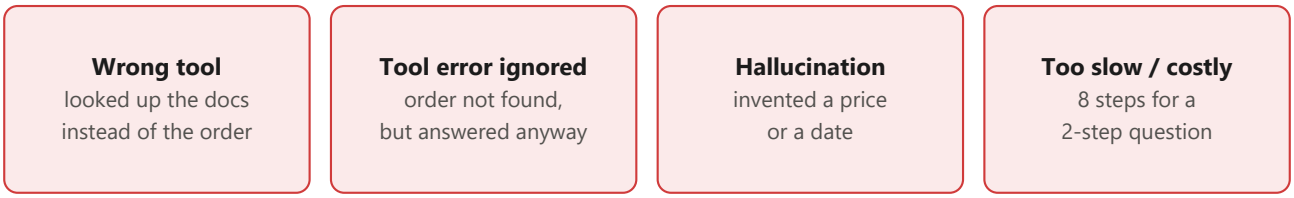


Figure 6 — Four silent failure modes of a support agent.

On top of that, agents **change all the time**. Developers tweak prompts, switch to a cheaper model, add a tool or change the step limit. Each change might make things better for some questions and worse for others. Without measurement, you only find out when a customer complains.

8. Idea one: observability (the flight recorder)

Observability means being able to understand what's happening inside a system by looking at the data it records. For agents, the key record is a **trace**: a detailed log of one run.

> **Everyday analogy**
> An aeroplane's flight recorder (the "black box") records altitude, speed and every control input throughout a flight. When something goes wrong, investigators replay it second by second. A trace is a flight recorder for one agent run.

A good trace answers these questions for any run, even weeks later:

- What did the user ask, and what was the final answer?
- Which model and which version of the agent handled it?
- What steps happened, in what order? Which tools were called, with which arguments, and what did they return?

- How long did each step take? How many tokens did each LLM call use? What did it cost?
- Did anything fail? Where, and with what error message?

In observability vocabulary, the whole run is the **trace** and each individual operation inside it is often called a **span**. AgentLens calls them **runs** and **steps**, which is easier to remember.

> **Why this matters**
> Without traces, debugging an agent means guessing. With a trace you can open the failing run and see that, for example, the model never called the order lookup and simply invented the delivery date. That points straight at the fix: a stronger instruction in the prompt.

9. Idea two: evaluation (grading the answers)

A trace tells you *what* happened. **Evaluation** tells you whether it was *good*. An **evaluator** is a function that looks at a run and returns a score, and in AgentLens always a pass or fail as well. There are two broad families:

Family	How it works	Strengths	Weaknesses
Rule-based	Plain code checks: did the answer contain the tracking number? Was the order tool called? Are all numbers in the answer present in the tool results?	Fast, free, perfectly consistent, easy to explain.	Rigid. Can't understand meaning, only patterns.
LLM-as-a-judge	A second model reads the question, the sources and the answer, and grades it using a written rubric.	Understands meaning, catches subtle errors, flexible.	Costs tokens, can itself be wrong, less consistent.

Good evaluation uses **both**: rules for what can be checked precisely, and a judge for what needs understanding. AgentLens grades four qualities of every run:

- **Task success**: did it do what the question needed? (rules)
- **Faithfulness, rules version**: are the facts in the answer backed by what the agent was shown?
- **Faithfulness, judge version**: does a second model agree the answer is grounded and complete?
- **Efficiency**: did it stay within a budget of steps, tokens and time?

10. Idea three: regression testing

In software, a **regression** is when something that used to work stops working after a change. **Regression testing** means re-running a fixed set of tests after every change to catch that.

For agents it works like this:

1. Write a **dataset** of test cases: realistic questions, each with a description of what a good answer must contain.
2. Run the whole dataset through **version 1** of the agent and record every result. This is an **experiment**, and it becomes your **baseline**.
3. Make a change (new prompt, cheaper model) to get **version 2**, the **candidate**, and run the same dataset.

4. **Compare**: success rate, quality scores, latency, tokens and cost, and which individual test cases flipped from pass to fail.

> **Key idea**
>
> Every change to an agent is a trade-off. Version 2 might be 40% cheaper and 10% faster, but fail 7% more questions. Regression testing doesn't decide for you. It puts the trade-off in front of you in numbers, before your users find it.

Those three ideas (observe, evaluate, compare) are the whole of AgentLens. Part 3 shows how each one is built.

PART 3

How AgentLens works

A guided tour through every part of the project. We'll follow one real question, "Where is my order 1042?", from the moment it's asked to the moment it appears, graded, on the dashboard.

11. The big picture

AgentLens is a single application (a **monolith**) made of three running pieces plus an external AI service:

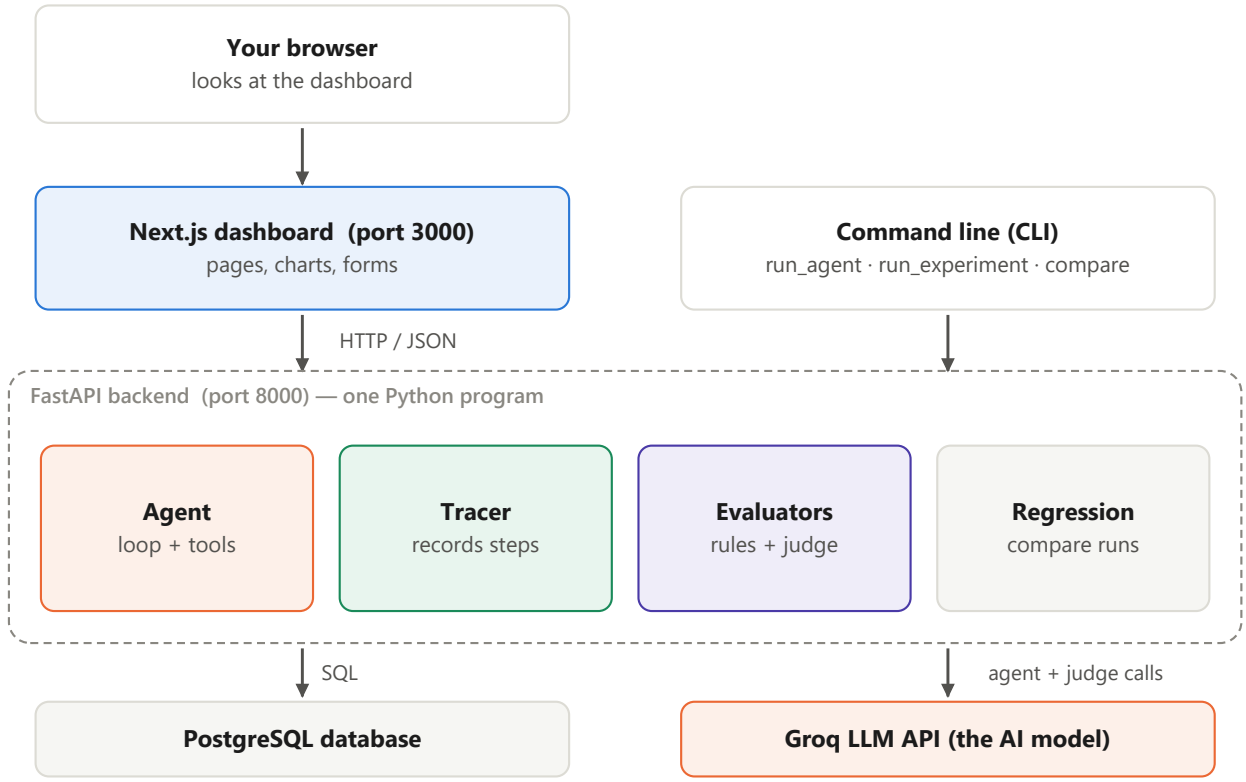
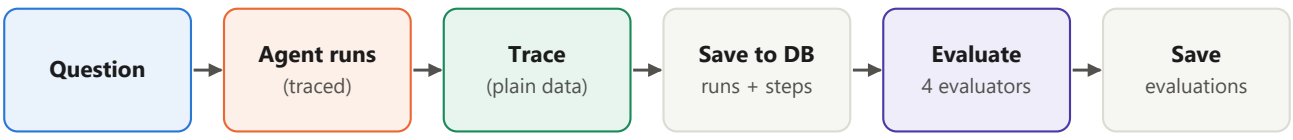


Figure 7 — AgentLens architecture.

Piece	Technology	Job
Backend	Python, FastAPI	Runs the agent, records traces, runs evaluators, compares experiments, and answers the dashboard's requests.
Database	PostgreSQL	Stores every run, step, evaluation and experiment permanently.
Dashboard	Next.js, Tailwind CSS, Recharts	Shows the data as pages, tables and charts in a web browser.
CLI scripts	Python	Let you run the agent or a full experiment from a terminal.
LLM provider	Groq API	Hosts the AI models (the agent's brain and the judge).

Every question that goes through AgentLens follows the same six-stage pipeline:



services/run_service.py → run_and_save() does all six stages for one question.

Figure 8 — The journey of one question.

> **Why this matters**
> Why not use microservices, message queues or Redis? Because nothing here needs them. One developer, one machine, 14 test cases run one after another. Extra infrastructure would add things that can break without teaching anything new about agents. Start simple, and add complexity only when a real problem demands it.

12. The sample world: ShopWise

To observe an agent we need an agent, and an agent needs a job. AgentLens includes a small **customer-support agent for a fictional online store called ShopWise**. It's intentionally simple, because the point of the project is the observation, not the agent.

The store's "database" is two small JSON files (`backend/agent/data/`):

- **orders.json**: five orders (1041–1045) in different states: delivered, shipped (with a tracking number), processing, cancelled.
- **knowledge_base.json**: seven help articles: returns (30 days, but only 15 for electronics), refunds (5–7 business days), shipping ($4.99, free over $50, express $12.99 in 2 days, US and Canada only), cancellations, warranty, damaged items and payment methods.

> **Demo data vs real data**
> This is **demo data**: invented, and kept separate from the application's real data. Nothing in the database is pre-filled. Every run, score and chart you see was produced by actually running the agent.

The frozen clock

The agent is told that today is **2026-09-25**, always. Why? The question "Can I still return the headphones from order 1041?" depends on the date. They were delivered on September 8 and electronics have a 15-day window, so on September 25 the answer is *no*. If we used the real date, the correct answer would change over time and our tests would start "failing" even though the agent hadn't changed. Tests must be **reproducible**.

13. The agent

Its three tools

Tool	Input	What it does
lookup_order	order_id	Returns the order's full record, or {"error": "Order 9999 not found."}
search_knowledge_base	query	Finds the two help articles sharing the most keywords with the query (a simple keyword search).
calculator	expression	Safely evaluates arithmetic like "15 * 2 + 34.50".

Two design choices in the tools are worth knowing:

- **Errors are returned as data, not raised as crashes.** If the model asks for order 9999, the tool returns an error message *to the model*, which can then politely tell the customer the order doesn't exist. A crash would end the whole run.
- **The calculator never uses Python's `eval()`.** The expression comes from the model, and a model could be tricked into sending malicious code. The calculator parses the expression and allows only numbers and `+ − × ÷`. There's a test that proves an attack string is rejected.

Two versions

A **version** is everything that can change the agent's behaviour, kept in `backend/agent/versions.py`:

	Version 1 (v1)	Version 2 (v2)
Model	openai/gpt-oss-120b (larger)	openai/gpt-oss-20b (smaller, ~half the price)
System prompt	Detailed rules: always use lookup_order, never guess, search the help articles before answering policy questions, use the calculator for maths, answer only from tool results, ask for a missing order ID	One sentence: "You are a helpful support assistant for ShopWise. Answer the customer briefly."
max_steps	6	4

v2 is the "cost-cutting" version that someone might plausibly ship. It will be cheaper and faster. Is it worse? Only measurement can say.

The loop, in code

```
for _ in range(config["max_steps"]):
    with tracer.step("llm", "llm_call", input={"messages": list(messages)}) as step:
        response = chat(model, messages, tools=TOOL_SCHEMAS)
        step.record_llm_response(model, response)
        step.name = "planner" if response.tool_calls else "final_answer"

    messages.append(response.assistant_message)
    if not response.tool_calls:
        return response.text, "success"            # plain text = final answer

    for call in response.tool_calls:
        with tracer.step("tool", call.name, input=call.arguments) as step:
            result = run_tool(call.name, call.arguments)
            step.output = result
        messages.append({"role": "tool", "tool_call_id": call.id,
                         "content": json.dumps(result)})

return "Sorry, I couldn't complete this request...", "max_steps"
```

Read it next to Figure 5: call the model, and if it asked for tools, run them and loop; otherwise return the answer. Notice the `with tracer.step(...)` lines. Those are the only additions needed to make the agent observable.

Walkthrough: "Where is my order 1042?"

#	What happens	Messages in the conversation
1	Program builds the conversation.	system (rules + date), user ("Where is my order 1042?")
2	**LLM call — planner.** The model sees the tools and decides to use one. It replies: call lookup_order({"order_id": "1042"}).	+ assistant (the tool request)
3	**Tool step.** Python runs lookup_order("1042") and gets: status shipped, tracking TRK-88213, estimated delivery 2026-09-27, items…	+ tool (the order JSON)
4	**LLM call — final answer.** The model reads the order data and writes: "Your order 1042 has shipped and should arrive by September 27. Tracking number: TRK-88213." (illustrative)	+ assistant (the answer)

Three steps: two LLM calls and one tool call. Every one of them is recorded.

14. Tracing: recording everything

The tracer lives in `backend/tracing/tracer.py` (about 100 lines). It has three parts:

- **Step**: a record of one LLM call or tool call.
- **Tracer**: collects steps while the agent runs. Its `step()` method is used with Python's `with` statement, which guarantees the step is recorded even if something crashes inside it.
- **Trace**: the finished record of the whole run, returned by `tracer.finish()`.

How the with-statement recorder works

```
@contextmanager
def step(self, step_type, name, input):
    step = Step(step_index=len(self.steps), step_type=step_type, name=name, input=input)
    start = time.perf_counter()                      # 1. start the stopwatch
    try:
        yield step                                   # 2. the agent's code runs here
    except Exception as e:
        step.error = f"{type(e).__name__}: {e}"      # 3. note any crash...
        raise                                        #    ...but don't hide it
    finally:
        if not step.latency_ms:                      # 4. stop the stopwatch
            step.latency_ms = int((time.perf_counter() - start) * 1000)
        self.steps.append(step)                      # 5. always keep the step
```

> **Everyday analogy**
> It works like a lab technician's logbook routine: before an experiment, write the time and what you're about to do. After it, write the result and the time, *even if the experiment blew up*. The finally block is the "even if it blew up" part.

What gets recorded

Level	Fields
Run	agent name and version, model, user input, final output, status (success / error / max_steps), error message, start and end time, total latency, total input and output tokens, estimated cost
Step	position in the run, type (llm or tool), name (planner, lookup_order, final_answer…), input (the exact messages sent, or the tool arguments), output (the model's reply, or the tool result), latency, tokens and cost (LLM steps), error

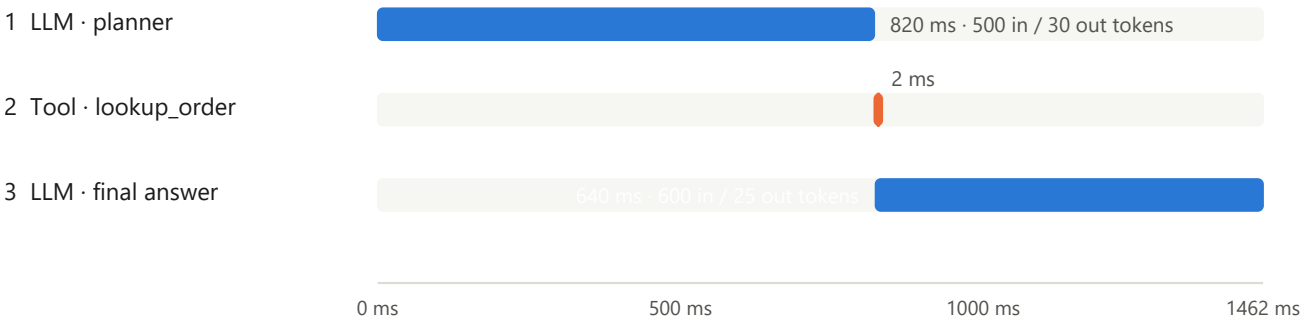


Figure 9 — A trace drawn as a timeline (illustrative timings). The dashboard shows this for every run.

Three subtle problems the tracer solves

1. Snapshots, not references. The messages list keeps growing during the run. If a step stored a *reference* to it, then by the end every step would appear to have seen the full conversation. The tracer stores `list(messages)`, a copy taken at that moment. A test checks that the planner step saw exactly 2 messages.

2. Rate limits distort time. When Groq says "too many requests, wait 20 seconds", the program waits and retries. If we timed the whole thing, the trace would say the model took 21 seconds, blaming the model for time spent waiting in line. `llm.py` times **only the successful attempt**, and the run's latency is the **sum of its steps**, so waiting is never counted. Without this, whichever version happened to hit the limit more would look slower, and the comparison would be unfair.

3. Crashes are runs too. If the API key is wrong or the network is down, `run_agent()` catches the exception and still returns a trace, with status *error* and the error message. A failed run that vanished without a record would be the worst kind of failure to debug.

Cost estimation

`backend/tracing/pricing.py` holds a small table of prices per million tokens. Each LLM step's cost is computed from its own token counts (the formula in section 2), and the run's cost is the sum. For an unknown model the cost is recorded as *empty* rather than a guess, because a missing number is honest and a wrong number is misleading. On Groq's free tier you actually pay nothing; the estimate shows what the run would cost on the paid tier.

15. Storing traces: the database

A trace in memory disappears when the program ends. To compare today's runs with last week's, we need to store them. A **database** is a program designed to store data safely and let you search it quickly. **PostgreSQL** ("Postgres") is a popular, free, reliable database that stores data in **tables**, like spreadsheets with strict columns.

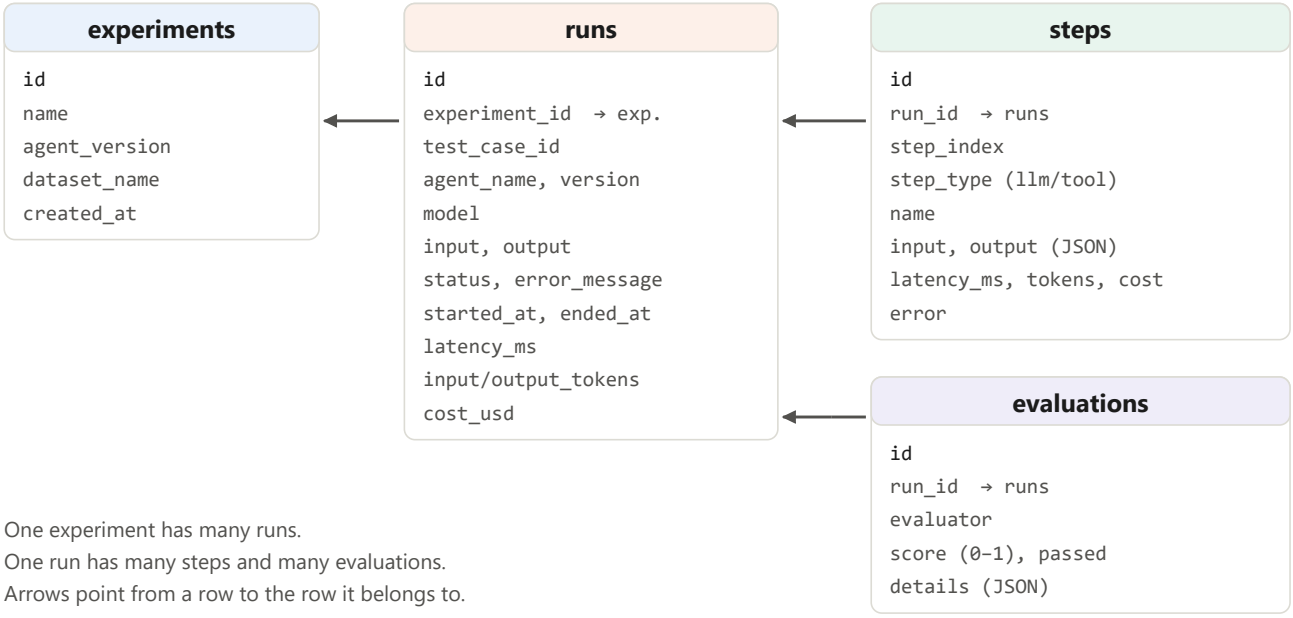


Figure 10 — The four tables and how they connect.

Tables connect through **foreign keys**: each step row has a `run_id` column holding the id of the run it belongs to. That's how the dashboard can fetch "run #11 and all its steps" in one go.

Design decisions

- **JSON columns for step input and output.** An LLM step's input is a list of messages; a tool step's input is arguments like {"order_id": "1042"}. They have different shapes, so they're stored as flexible JSON.
- **No stored averages.** Numbers like "average latency of experiment #2" are calculated from the runs every time they're requested. If they were stored, they could go stale when data changed. With 14 runs per experiment, calculating is instant.
- **Runs have a readable integer id** (#1, #2, #1024…) so you can say "look at run 1024".
- **Field names in the Trace match the table columns exactly.** So evaluators work on a fresh trace or on a run loaded from the database, without any conversion.

The code talks to the database through **SQLAlchemy**, an **ORM** (Object-Relational Mapper). An ORM lets you work with rows as ordinary Python objects: `run.steps` gives you the list of steps. SQLAlchemy writes the SQL for you. `database/save.py` turns a Trace into a Run with Step children and saves them in one transaction, so either everything is saved or nothing is.

16. The evaluators, one by one

Every evaluator returns the same shape: a name, a **score from 0.0 to 1.0**, **passed** (true/false) and **details** explaining why. A common 0–1 scale means different evaluators can be averaged and compared.

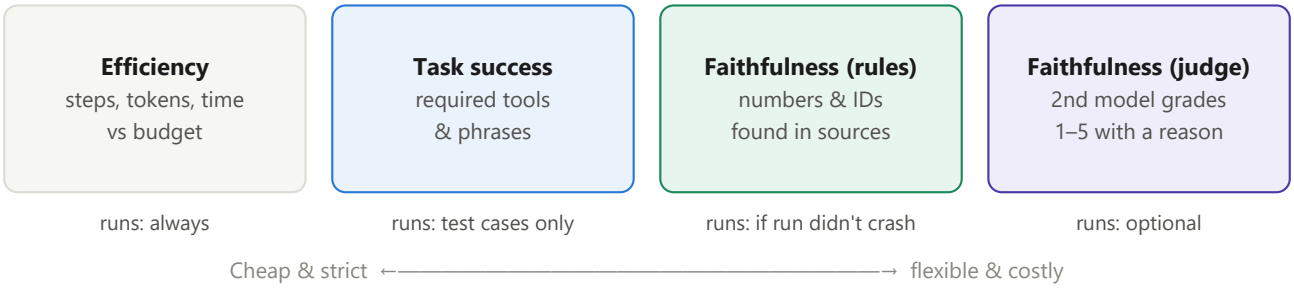


Figure 11 — Four evaluators, from cheap and strict to flexible and costly.

Efficiency — did it stay within budget?

Checks three numbers against limits: steps ≤ 6, tokens ≤ 6,000 and latency ≤ 10 seconds by default. A test case can set a tighter budget. For "Where is my order?" (no ID given) the budget is 2 steps, because the right move is simply to ask for the order number. Score = checks passed ÷ 3.

Task success — did it do the job?

Uses the expectations written into each test case. Example:

```
{ "id": "order-status-shipped",
  "input": "Where is my order 1042?",
  "expected": {
    "must_call_tools":  ["lookup_order"],
    "must_contain":     ["TRK-88213"],
    "must_contain_any": ["shipped", "on its way", "in transit"] } }
```

Each rule becomes a check, plus an automatic check that the run didn't crash:

Check	Good answer (v1)	Sloppy answer ("Your order was delivered yesterday.")
run completed	✓	✓
called lookup_order	✓	✗ answered without looking
mentions 'TRK-88213'	✓	✗
mentions shipped / on its way / in transit	✓	✗
Result	**4/4 = 1.00 — pass**	**1/4 = 0.25 — fail**

The run only counts as a success if **every** check passes. Case-insensitive text matching keeps it forgiving about capital letters, and the "any of" lists allow different phrasings.

Faithfulness (rules) — are the facts backed by sources?

First, gather the **sources**: everything the agent was allowed to know, meaning the system prompt, the user's question and every tool result. Then pull the checkable **facts** out of the answer: numbers, prices, dates and tracking numbers (single digits like "2" are ignored as too common). Each fact must appear somewhere in the sources.

```
Answer:  "Order 1042 shipped and arrives 2026-09-27. Tracking TRK-88213. Total $64.50."

Facts found:  1042  2026  09  27  64.50  TRK-88213
In sources?   yes   yes   yes yes  NO    yes        → 5 of 6 = 0.83  → pass (≥ 0.75)
```

Why is 64.50 flagged? The order has 2 T-shirts at $15 and a backpack at $34.50, which comes to $64.50, but no tool ever returned 64.50. The model did the maths in its head instead of using the calculator. Here it happened to be right, but the rule has correctly spotted an *unverified* number. A made-up price like $99.99 would be flagged in exactly the same way.

> **Watch out**
> This evaluator can't catch a wrong claim that contains no number, like "your order was delivered" when it actually shipped. And it matches loosely: "27" could appear in a source for an unrelated reason. That's why we also have a judge.

Faithfulness (LLM judge) — what does a second model think?

The judge (gpt-oss-120b by default) receives a prompt containing the sources, the question and the answer, plus a rubric:

```
5 = every claim is supported by the sources and the question is fully answered
4 = supported by the sources, small omissions
3 = mostly supported, but one claim is unsupported or the answer is incomplete
2 = several unsupported or wrong claims
1 = mostly made up, contradicts the sources, or ignores the question
Reply with JSON only: {"score": <1-5>, "reason": "<one or two sentences>"}
```

The 1–5 score is converted to 0–1 with (score − 1) ÷ 4, so 5 → 1.0, 4 → 0.75 and 3 → 0.5. A score of 4 or more passes. The judge's **reason** is saved and shown on the dashboard, so you can check its work. Its own token usage and cost are also recorded, because **evaluation isn't free either**.

Design details: temperature 0 and a fixed rubric keep grading consistent. The reply is parsed defensively (models sometimes wrap JSON in extra text). If the judge fails, that's recorded as an evaluator error, and those results are left out of averages because they say nothing about the agent.

> **Key idea**
> **Which evaluators run when?** Efficiency: always. Task success: only for test-case runs, because it needs to know what "correct" means. Faithfulness: only if the run didn't crash, since grading an empty answer is meaningless. Judge: optional, switched off with `--no-judge` to save tokens.

17. The test dataset and experiments

`backend/datasets/support_v1.json` contains **14 test cases** covering the situations a support agent must handle:

Category	Test cases
Orders (7)	shipped order with tracking · processing order · cancelled order · order that doesn't exist (9999) · question with no order ID · order total including shipping (needs arithmetic) · cancelling an already-shipped order
Policy (7)	electronics return window · returning a specific order (needs two tools and date reasoning) · express shipping price and speed · free-shipping threshold · refund timing · international shipping · paying with Bitcoin

Good test sets include **easy cases** (single lookup), **hard cases** (multiple tools, reasoning), and **trick cases** (missing information, things that don't exist, things the store doesn't do). The trick cases catch agents that always produce *an* answer instead of the *right* one.

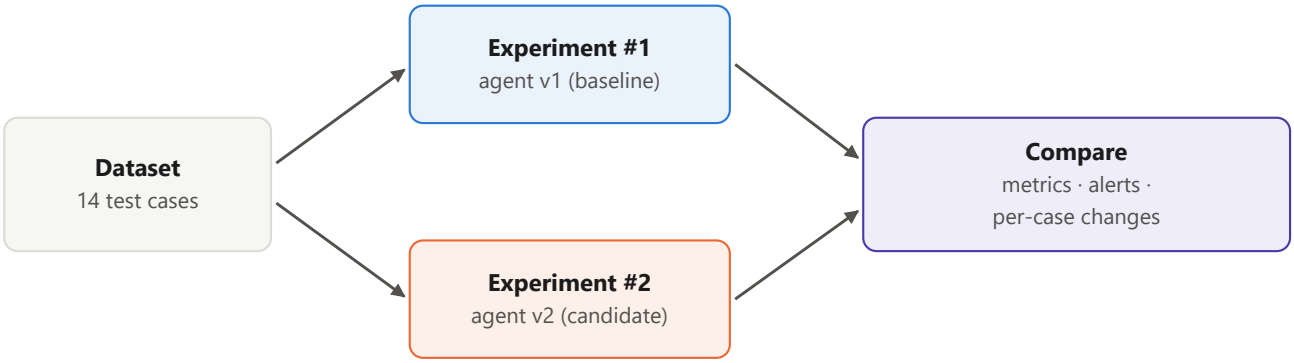


Figure 12 — An experiment runs the whole dataset through one agent version.

An **experiment** is one pass of the dataset through one version. `services/experiment_service.py` creates an experiment row, then for each test case runs the full pipeline (agent → trace → save → evaluate), tagging each run with the experiment id and the test case id. It pauses 2 seconds between cases to stay under the free-tier limits.

18. Regression detection

`services/regression.py` turns stored runs into a verdict. First it **summarises** each experiment:

Metric	How it's calculated
Task success rate	% of runs whose task_success evaluation passed
Avg eval score	average of each evaluator's average score (so every evaluator counts equally)
Failed runs	% of runs that ended in error or hit max_steps
Avg latency / tokens / steps	simple averages across runs
Avg and total cost	from each run's estimated cost

Then it **compares** baseline A with candidate B and raises an alert when B is worse by more than a threshold:

Alert	Threshold
Task success dropped	by 5 percentage points or more
Average eval score dropped (overall, or for any single evaluator)	by 0.05 or more (on the 0–1 scale)
Average latency increased	by 20% or more
Average cost per run increased	by 20% or more
Failed-run rate increased	any increase

Percentage points vs percent

If task success goes from 86% to 79%, it dropped by **7 percentage points** (86 − 79). As a *relative* change it dropped by 8.1% (7 ÷ 86). Mixing these up is a classic reporting mistake, so AgentLens reports rates in **points** and quantities like latency and cost in **percent**.

Why averages aren't enough

Imagine v2 fixes the "Bitcoin" case but breaks the "cancel shipped order" case. The success rate stays exactly the same, so an average-only report would say "no change", yet a real behaviour broke. AgentLens therefore also compares **test case by test case**: for every case and every evaluator, did it go pass → fail (*regressed*) or fail → pass (*improved*)? Each entry links to the two runs, so you can open them side by side.

An illustrative comparison:

```
#1 v1 baseline (14 runs)          #2 v2 candidate (14 runs)
  task success : 85.7%              task success : 78.6%
  eval score   : 0.91               eval score   : 0.84
  avg latency  : 1420 ms            avg latency  : 690 ms
  avg cost     : $0.000210/run      avg cost     : $0.000060/run

Alerts:
  ! Performance regression detected: task success decreased by 7.1 percentage points.
  ! Quality regression: average evaluation score dropped by 0.07.
  worse:  return-order-1041     task_success      run #9 -> #23
  worse:  order-total           faithfulness_rules run #6 -> #20
```

(Illustrative numbers.) The verdict is a trade-off in plain sight: v2 is about 3.5× cheaper and twice as fast, but fails an extra question, and the report tells you exactly which one.

19. The backend API (FastAPI)

The dashboard needs data, and the backend provides it through an **API**: a set of URLs (**endpoints**) that return data as JSON instead of web pages. **FastAPI** is a Python framework for building APIs. You write a normal function, add a decorator like `@router.get("/api/runs/{run_id}")`, and FastAPI handles the web part. It also generates an interactive documentation page at `/docs` where you can try every endpoint.

Method + URL	Returns
GET /api/health	{"status": "ok"} — is the server alive?
GET /api/stats	everything the Overview page needs: totals, averages, recent runs, trend, latest regression
GET /api/runs	a list of runs (optionally only one experiment's)
GET /api/runs/{id}	one run with all its steps and evaluation details
POST /api/runs	runs the agent on a new question, saves and evaluates it, returns the result
GET /api/experiments	every experiment with its summary metrics
GET /api/experiments/compare?a=1&b=2	the full comparison report
GET /api/versions	the available agent versions and their models

GET requests read data. **POST** requests create something. The POST endpoint is written as a plain `def` (not `async def`) on purpose: FastAPI then runs it on a separate worker thread, so while the agent waits several seconds for the AI model, the server can still answer other requests.

20. The dashboard (Next.js)

The dashboard is a website built with **Next.js** (a framework for React, the most popular way to build web interfaces), styled with **Tailwind CSS**, with charts drawn by **Recharts**.

Page	What you see
Overview /	Six stat tiles (total runs, run success rate, task success, avg latency, avg cost, avg eval score); the latest regression check with its alerts; trend charts across experiments; a "Try the agent" form; recent runs.
Runs /runs	Every run with status, latency, tokens, cost and a pass/fail badge per evaluator. Can be filtered to one experiment.
Run details /runs/[id]	The flight recorder: user input → each step (with a timing bar, tokens, cost, tool arguments and responses, and the exact prompt sent) → final answer → every evaluation with its reasons.
Experiments /experiments	A table of every experiment's summary metrics, and a picker to compare two.
Compare /compare	Side-by-side metrics with better/worse markers, the alerts, and lists of cases that got worse or better.

How a page gets its data

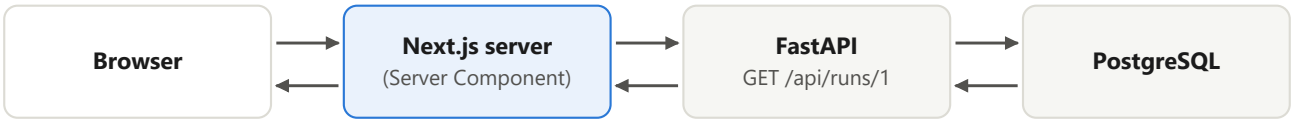


Figure 13 — Data flow for one page view.

Pages are **Server Components**: they run on the Next.js server, fetch data from FastAPI, and send finished HTML to the browser. The "Try the agent" form uses a **Server Action**, a function that runs on the server when the form is submitted. It calls the backend, then redirects you to the new run's page. Because the browser never contacts FastAPI directly, there's no need for extra cross-origin (CORS) security configuration.

> **Key idea**
> Every number on the dashboard comes from the database through the API. Nothing is hard-coded. If there are no runs yet, the page says so and tells you which command to run.

Design details: pass/fail is always shown with an icon and a word (✓ pass, ✗ fail), never by colour alone, so colour-blind users can read it. The chart colours were checked with a colour-blindness validator. Light and dark mode are both supported.

21. Docker: packaging it up

Running AgentLens by hand means installing Python, Node.js and PostgreSQL at the right versions. **Docker** packages each piece with everything it needs into a **container**, a lightweight, isolated mini-computer that runs the same way on any machine.

> **Everyday analogy**
> Shipping containers standardised global trade: whatever is inside, every crane, ship and truck handles the box the same way. A Docker container does the same for software.

An **image** is the recipe (a Dockerfile describes how to build it), and a **container** is a running copy. **Docker Compose** starts several containers together from one file, `docker-compose.yml`:

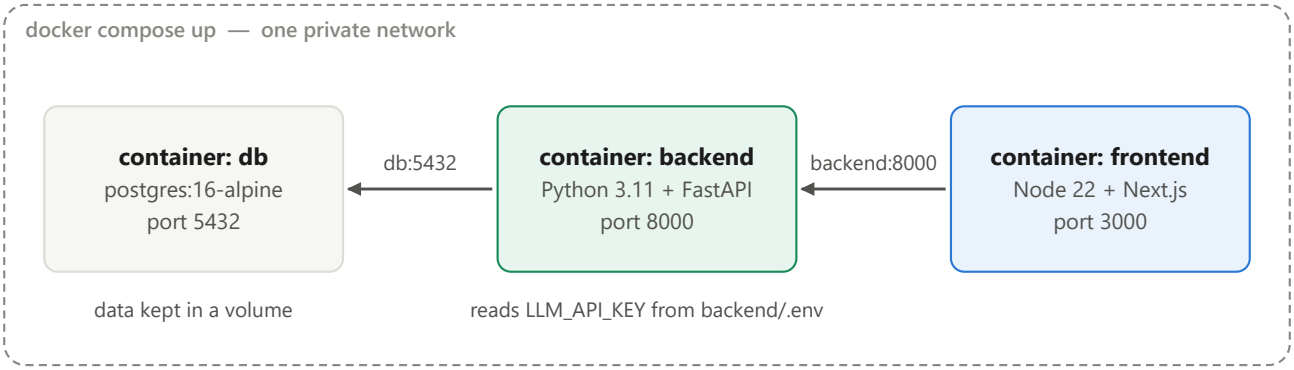


Figure 14 — The three containers started by docker compose up.

- Inside Docker, "localhost" means the container itself, so containers reach each other by service name: the backend connects to `db:5432`, and the dashboard to `backend:8000`.
- The backend waits until Postgres reports healthy before starting.
- The frontend image is built in two stages: build with all tools, then copy only the small standalone server into the final image.
- Database files live in a named **volume**, so data survives restarting the containers.

> **Watch out**
> Docker Desktop on Windows runs a small Linux virtual machine that wants 2 GB or more of RAM. On an 8 GB laptop, cap it (a `.wslconfig` file with `memory=3GB`), close heavy apps, or use a free hosted Postgres (Neon) for daily development and Docker only for the final packaging test.

22. Testing without an AI

AgentLens has **19 automated tests** (run with `pytest`). But how do you test code whose main ingredient, the AI, is slow, costs money, is rate-limited and doesn't give the same answer twice?

Answer: in tests, the real `chat()` function is replaced by a **fake LLM**, a short scripted function in `tests/conftest.py`. It pretends to be a *careful* agent when asked as gpt-oss-120b (looks up the order, answers from it) and a *sloppy* one as gpt-oss-20b (answers immediately with a made-up price). Tests also use a throwaway **SQLite** database file instead of Postgres. This technique is called **mocking**.

Test file	What it proves
test_tools.py (7)	orders are found, missing orders return errors, the search finds the right article, the calculator does maths and rejects code
test_tracing.py (5)	a full run produces planner → lookup_order → final_answer steps with correct tokens and cost; snapshots don't change; crashes become error runs; costs are right
test_evaluators.py (6)	task success passes the careful agent and fails the sloppy one; faithfulness flags the invented $99.99; budgets work; judge replies are parsed
test_regression_and_api.py (1)	end to end: two experiments are run and stored, the regression is detected, and the API serves stats, run details and the comparison

> **Why this matters**
>
> The fake LLM is not "fake functionality". It exists only inside the tests folder and is never used by the real application. It lets us check that *AgentLens* works correctly (tracing, storage, grading, comparison) quickly and for free, separately from whether the *AI* is any good.

PART 4

Using AgentLens

Setting it up on your machine, the everyday loop of improving an agent, how to read what the dashboard tells you, and what to do when something goes wrong.

23. Setting it up

You need **Python 3.11+** and **Node.js 20+**. Docker is optional.

Step 1 — get a free AI key

Sign up at console.groq.com and create an API key. In the `backend` folder, copy `.env.example` to `.env` and paste the key after `LLM_API_KEY=`. The `.env` file holds secrets, so it's listed in `.gitignore` and never committed to Git.

Step 2 — get a PostgreSQL database (choose one)

Option	How	Best for
Neon (hosted, free)	Create a project at neon.tech and paste its connection string into DATABASE_URL	low-RAM laptops; zero install
Docker, database only	docker compose up db (the default DATABASE_URL already matches)	if Docker runs comfortably
Native install	Install PostgreSQL; create user, password and database named agentlens	working offline

Step 3 — backend

```
cd backend
python -m venv .venv
.venv\Scripts\activate              (macOS/Linux: source .venv/bin/activate)
pip install -r requirements.txt
pytest                               19 tests, needs no key and no database
uvicorn main:app --reload            API on http://localhost:8000  (docs at /docs)
```

Step 4 — dashboard (in a second terminal)

```
cd frontend
npm install
npm run dev                          dashboard on http://localhost:3000
```

Step 5 — generate real data

```
cd backend
python -m scripts.run_agent "Where is my order 1042?"            just print the trace
python -m scripts.run_agent "Where is my order 1042?" --save     store + evaluate it
python -m scripts.run_experiment --version v1                    experiment #1
python -m scripts.run_experiment --version v2 --compare-to 1     experiment #2 + comparison
```

> **Try it yourself**
>
> Run `python -m scripts.run_experiment --version v1 --limit 3 --no-judge` first. It runs only 3 test cases without the judge: a quick, cheap check that everything is connected.

24. The everyday workflow

AgentLens turns improving an agent from guesswork into a loop:

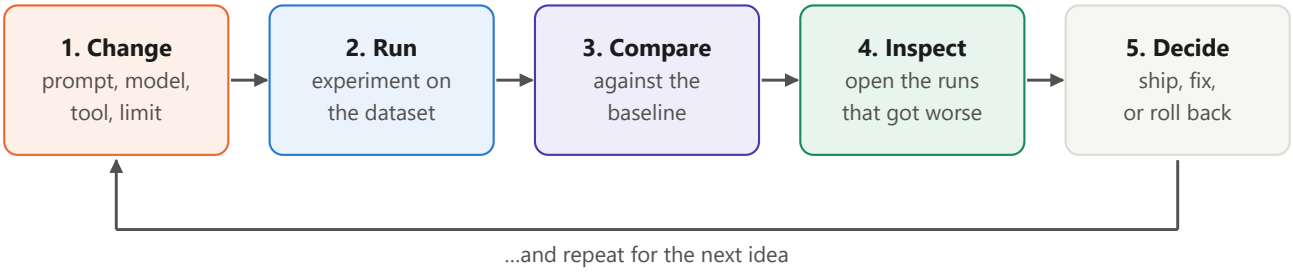


Figure 15 — The improvement loop.

A worked story (illustrative)

1. You want to save money, so you create v2 with the smaller model and a shorter prompt.
2. You run the dataset on v2 and compare with v1. The report says cost fell by 70% and latency by 50%, but **"Performance regression detected: task success decreased by 7.1 percentage points."**
3. The regressed list shows `return-order-1041`. You open v2's run for it.
4. The step timeline shows the problem at once: v2 called `lookup_order` but never `search_knowledge_base`. It guessed the return window was 30 days, and the task-success check "mentions '15'" failed.
5. You add one line to v2's prompt: "Check the help articles before answering policy questions." Re-run and compare. The case passes again, and v2 keeps most of its savings.

That's the whole value of the project in five steps: the regression was **detected** automatically, **located** down to one test case, **explained** by the trace, and the fix **verified** by re-running.

25. How to read a run

When you open a run's page, go through it in this order:

1. **Status badge.** *success* means it finished normally. *error* means something crashed (the message is shown). *max_steps* means it looped too long without answering.
2. **The evaluations at the bottom.** Which ones failed? Their details say exactly which check failed, which numbers were unsupported, or why the judge gave a low score.
3. **The tool steps.** Were the right tools called with the right arguments? Did a tool return an error the agent ignored?
4. **The planner step.** Open "Prompt sent to the model" to see exactly what the model saw. Missing information there often explains a wrong decision.
5. **The timing bars and tokens.** Which step was slowest or used the most tokens? That's where to optimise.

> **Key idea**
>
> Failures usually fall into one of three buckets: the model **didn't use a tool it should have** (fix: prompt), the **tool gave bad or missing data** (fix: the tool or the data), or the model **had the right data but misread it** (fix: prompt, or a stronger model). The trace tells you which bucket.

26. Troubleshooting

Symptom	Cause	Fix
Run fails with "LLM_API_KEY is not set"	No backend/.env, or the key line is empty	Create .env from .env.example and add your Groq key
"rate limited, waiting 20s…" in the terminal	Free tier: 8K tokens per minute	Normal. It retries automatically, and waiting isn't counted as latency
Dashboard shows "Couldn't load this page"	The backend isn't running	Start uvicorn in the backend folder
"connection refused" to the database	Postgres isn't running, or DATABASE_URL is wrong	Start the database; check the URL in .env
Judge evaluation shows an error	The judge replied without valid JSON, or hit a limit	Re-run; errored judgments are excluded from averages
A correct-looking answer fails task success	The test's phrase lists are too strict	Add the alternative phrasing to must_contain_any

> **Watch out**
>
> The last row matters: **evaluators can be wrong too.** When a result surprises you, read the run before trusting the score. Improving the test cases is part of the work.

PART 5

Reference

Honest limitations, ideas for going further, how AgentLens relates to professional tools, a glossary of every term in this guide, and a map of the code.

27. Limitations

AgentLens is a learning project. Being clear about what it *doesn't* do is part of understanding it:

- **Single runs per test case.** Because models are non-deterministic, a serious evaluation would run each case several times and report the variation. A 7-point drop on 14 cases is exactly one question, which could be noise.
- **Small dataset.** 14 cases is enough to demonstrate the idea, not to certify an agent.
- **Rule-based faithfulness only checks numbers and IDs**, and matches them loosely.
- **The judge isn't validated.** We don't measure how often it agrees with a human grader.
- **Cost is an estimate** from a hand-maintained price table.
- **No schema migrations.** Tables are created automatically but not upgraded if their design changes.
- **One built-in agent.** Other agents can't send traces to AgentLens yet.
- **No login.** Anyone who can reach the dashboard can see and create runs, which is fine on your own machine only.

28. Future improvements

Idea	What it would teach
Run each case N times; show pass rate and spread	statistics of non-deterministic systems
Re-evaluate old runs after changing an evaluator	separating data collection from grading
Compare the judge against human labels	evaluating the evaluator (judge calibration)
A small tracing SDK so any agent can send traces over HTTP	instrumentation and API design
Start experiments from the dashboard with a background job	background work and progress reporting
Alembic migrations	evolving a database safely
Embedding-based search for the knowledge base	retrieval-augmented generation (RAG)

29. How this relates to professional tools

Commercial and open-source platforms such as LangSmith, Langfuse and Arize Phoenix solve the same problem at much larger scale. The concepts carry over directly:

AgentLens	General industry term
Run	trace
Step	span / observation
Tracer + with tracer.step()	instrumentation (often automatic via SDKs or OpenTelemetry)
Test cases file	dataset / golden set
Experiment	experiment / evaluation run
Evaluators (rules, judge)	evaluators / scorers / graders
Compare page + alerts	regression testing, often run in CI before deploying

Having built each piece by hand, you know what those tools do under the hood, and can make sense of their dashboards and trade-offs.

30. Glossary

Term	Meaning
Agent	An LLM running in a loop that can call tools to take actions or fetch information.
API	A way for programs to talk to each other, usually over HTTP with JSON data.
Baseline / candidate	In a comparison: the known reference version (baseline) and the new version being judged (candidate).
Context window	The maximum number of tokens a model can read in one call.
Container (Docker)	An isolated, packaged, running copy of a program with everything it needs.
Dataset	A fixed list of test cases used to evaluate an agent.
Endpoint	One URL of an API, e.g. GET /api/runs/{id}.
Evaluator	A function that grades a run and returns a score, pass/fail and details.
Experiment	One run of a whole dataset through one agent version.
Faithfulness / groundedness	Whether an answer's claims are supported by the sources the agent was given.
Foreign key	A database column that points to a row in another table (steps.run_id → runs.id).
Hallucination	A confident but false statement produced by a model.
HTTP 429	The "Too Many Requests" error returned when a rate limit is exceeded.
JSON	A text format for structured data: {"order_id": "1042"}.
JSON schema	A JSON description of what inputs a tool accepts, sent to the model with the tool list.
Latency	How long something takes, here in milliseconds (ms). 1,000 ms = 1 second.
LLM	Large Language Model: an AI trained to predict the next token of text.
LLM-as-a-judge	Using a (usually stronger) LLM with a rubric to grade another model's output.
max_steps	The safety limit on how many times the agent loop may run.
Mocking	Replacing a real component (like the AI) with a scripted fake during tests.
Monolith	An application built and run as one program rather than many small services.
Non-deterministic	Able to give different outputs for the same input.
Observability	Understanding a system's internal behaviour from the data it records.
ORM	Object-Relational Mapper: lets code treat database rows as objects (SQLAlchemy here).
Parameters / weights	The billions of learned numbers inside a model.
Percentage point	The plain difference between two percentages (86% → 79% is 7 points).
Prompt	The input sent to a model: here, a list of role-tagged messages.

Term	Meaning
Provider	A company that hosts models and sells access through an API (Groq, OpenAI, Anthropic…).
Rate limit	A cap on requests or tokens per minute/day imposed by a provider.
Regression	A change that makes something worse that used to work.
Server Component / Server Action	Next.js code that runs on the server: to render a page, or to handle a form.
Span	Industry term for one operation inside a trace (AgentLens calls it a step).
System prompt	The developer's standing instructions to the model.
Temperature	A setting controlling randomness in the model's choices; 0 = most predictable.
Token	A chunk of text (word or word-piece) that models read and write, and that providers bill by.
Tool / tool calling	A function the model can ask your program to run; the request-run-return process.
Trace	The complete record of one agent run and all its steps.
Volume (Docker)	Storage that outlives a container, used here for database files.

31. Code map and command cheat sheet

File	What's inside
backend/llm.py	the only code that calls the AI provider; rate-limit retries; token counts
backend/agent/support_agent.py	the agent loop, wrapped in tracer steps; the frozen store date
backend/agent/tools.py	the three tools and their JSON schemas
backend/agent/versions.py	v1 and v2: model, system prompt, max_steps
backend/tracing/tracer.py	Step, Trace and Tracer (the flight recorder)
backend/tracing/pricing.py	price per million tokens; cost estimate
backend/database/models.py	the four tables
backend/database/save.py	trace → rows
backend/evaluators/*.py	task_success, faithfulness (rules), llm_judge, efficiency
backend/services/run_service.py	the six-stage pipeline for one question
backend/services/experiment_service.py	running a dataset as an experiment
backend/services/regression.py	summaries, thresholds, alerts, per-case comparison
backend/routes/*.py	the API endpoints
backend/datasets/support_v1.json	the 14 test cases
backend/tests/	19 tests with a fake LLM and SQLite
frontend/app/	the five pages and the Server Action
frontend/components/	tables, step timeline, evaluation cards, charts, forms
docker-compose.yml	db + backend + frontend containers

Command (from backend/)	Does
pytest	run all tests
uvicorn main:app --reload	start the API
python -m scripts.run_agent "..." [--version v2] [--save] [--json] [--no-judge]	run one question
python -m scripts.run_experiment --version v1 [--limit N] [--no-judge]	run the dataset
python -m scripts.run_experiment --version v2 --compare-to 1	run and compare
python -m scripts.compare 1 2	compare two existing experiments
docker compose up --build (from the project root)	start everything in Docker

What you now understand

If you've read this far, you can explain every one of these, and point to where it happens in real code:

- An LLM predicts the next token. It has no memory and no access to your data unless you provide it.
- Tokens drive cost, speed and limits, so they're worth measuring on every call.
- An agent is an LLM plus tools in a loop, with a step limit.
- A trace records every step of a run, including failures, and must measure time honestly.
- Evaluation layers cheap, strict rules with a flexible, costly judge, and neither is perfect.
- Regression testing runs a fixed dataset on two versions, compares both the averages and the individual cases, and turns every agent change into a visible trade-off.

> **The one-sentence summary**
>
> Observe what happened. Evaluate whether it was good. Compare before you ship. That is AgentLens, and it is also how professional teams make AI agents trustworthy.